Multilingual Speech Translation Assistant

---

1. Introduction

1.1 Purpose

This document specifies the requirements for the **Multilingual Speech Translation Assistant (MSTA)** — a system that converts spoken input into translated speech/text output using:

- **Whisper (Speech-to-Text)**
- **Transformer / Seq2Seq with Attention (Language Translation)**
- **Text-to-Speech (TTS) Engine**
- Modern AI deployment platforms

1.2 Scope

The system will:

1. Capture speech input from user
2. Convert speech → text using **Whisper**
3. Translate text using **Transformer / Seq2Seq with Attention**
4. Convert translated text → speech
5. Deliver output in real-time or near real-time
6. Deploy via cloud or web/mobile platforms

---

2. Overall Description

2.1 Product Perspective

The system consists of 3 core AI modules:

User Speech

↓

[Whisper STT]

↓

Source Text

↓

[Transformer Translation Model]

↓

Target Text

↓

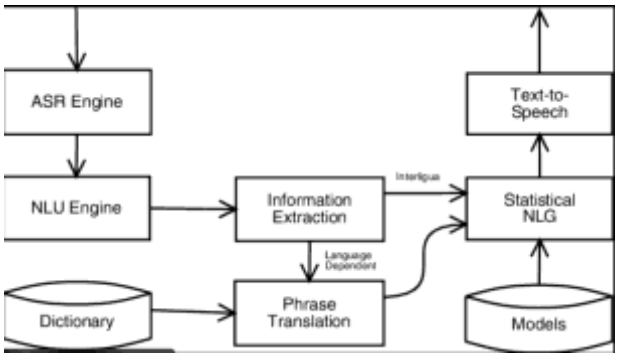
[TTS Engine]

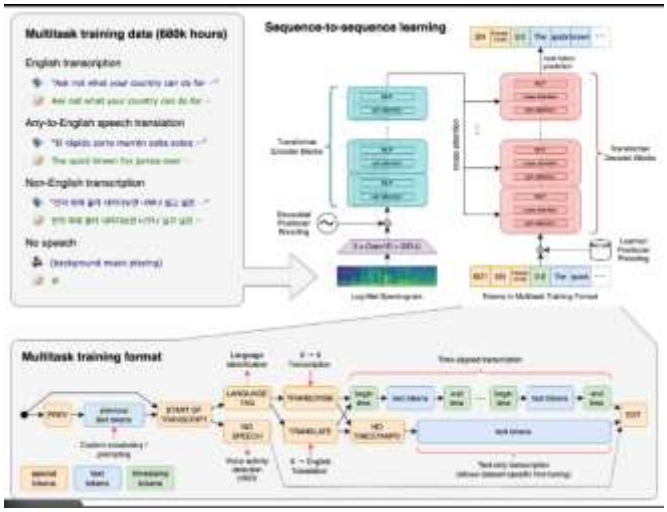
↓

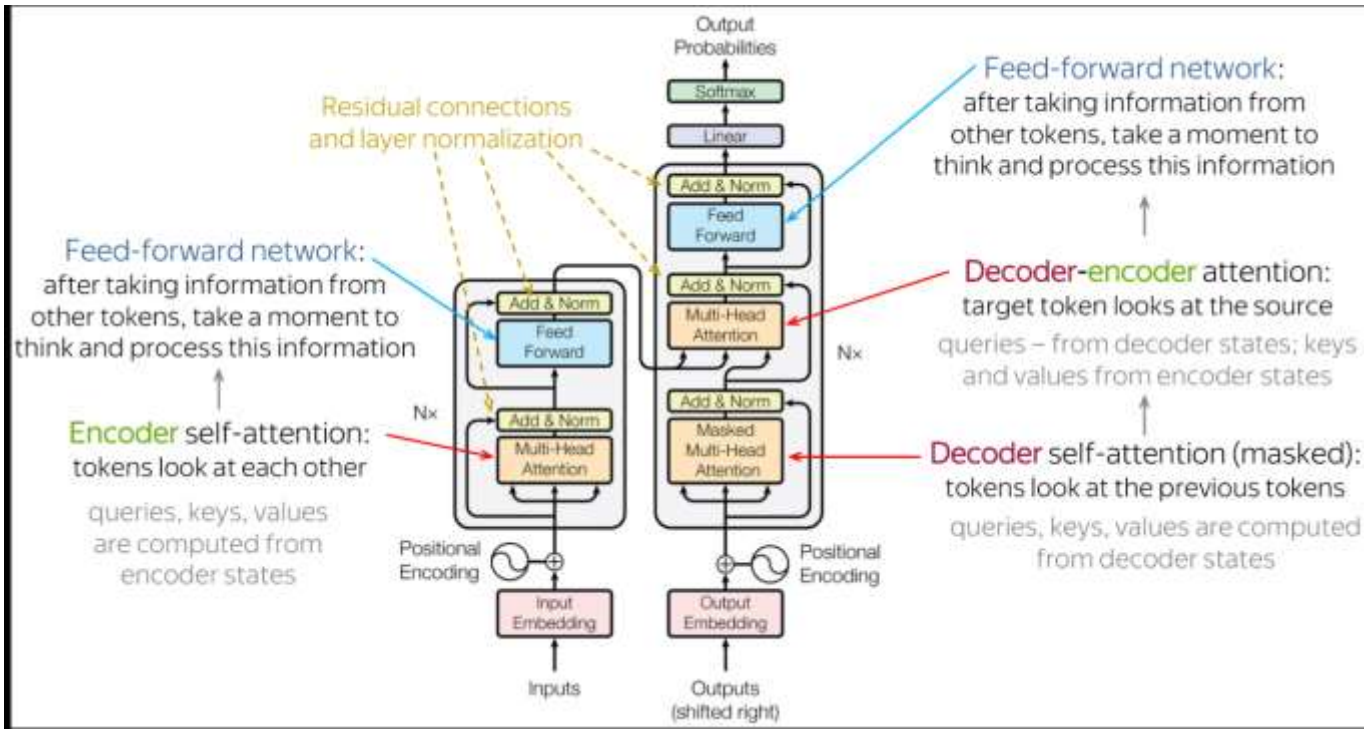
Translated Speech Output

3. System Architecture

3.1 High-Level Architecture







3.2 Architectural Components

Layer	Component	Technology
Frontend	Web/Mobile UI	React / Flutter
Backend API	REST / FastAPI	Python
STT Module	Whisper	PyTorch
Translation	Transformer / Seq2Seq w/ Attention	PyTorch / TensorFlow
TTS	Coqui TTS / gTTS	Python
Database	Logs / User Data	PostgreSQL / MongoDB
Deployment	Cloud / Container	Docker + Kubernetes

4. Functional Requirements

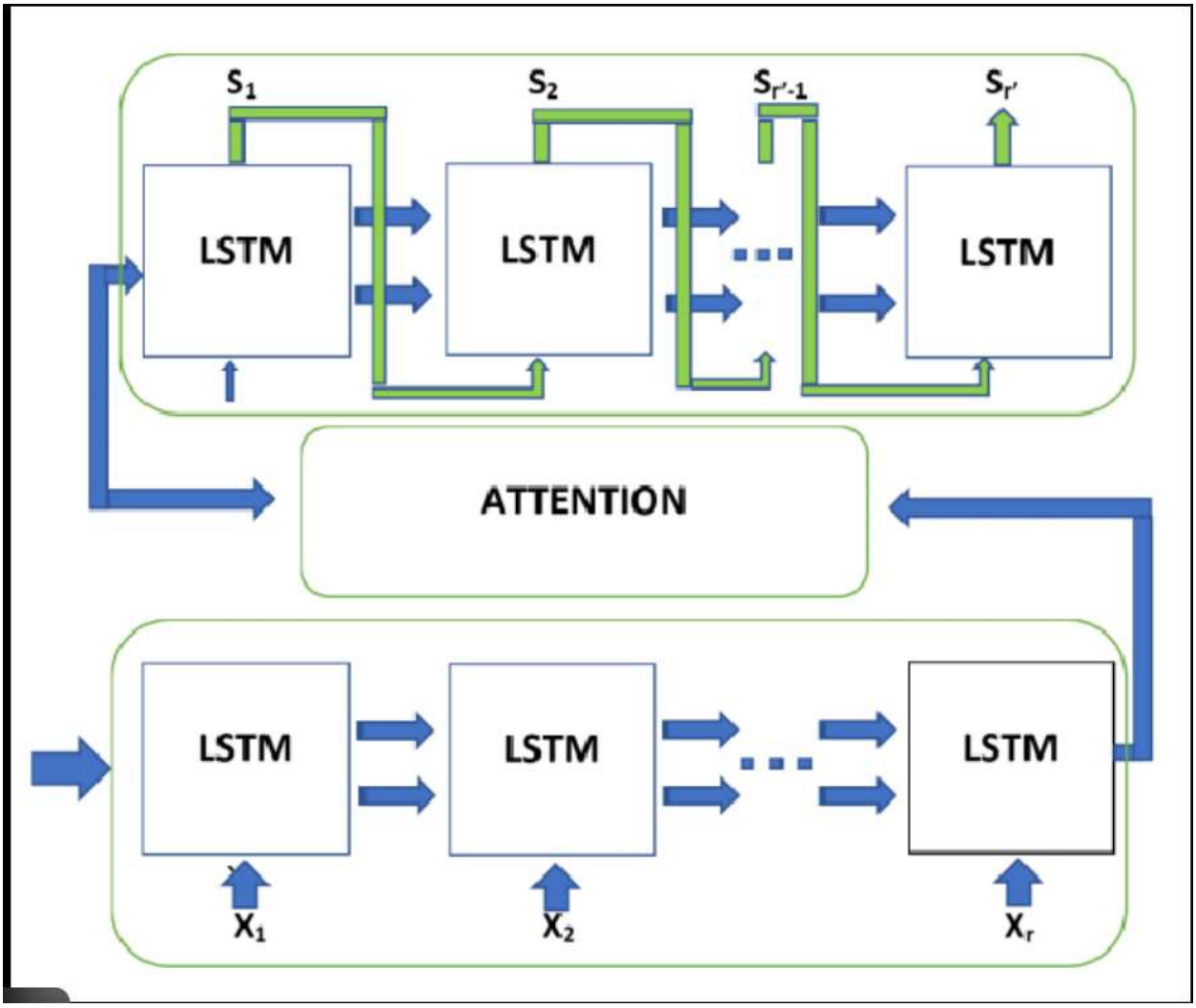
4.1 Speech-to-Text Module

FR-1: System shall capture audio from microphone.
FR-2: System shall transcribe speech using **OpenAI Whisper model**.
FR-3: System shall support multilingual input.
FR-4: Transcription accuracy ≥ 90% in supported languages.

Libraries:

- openai-whisper
- torch
- librosa
- sounddevice

4.2 Translation Module (Transformer / Seq2Seq with Attention)



FR-5: System shall translate text using Transformer architecture.

FR-6: Model shall support at least 20 languages.

FR-7: System shall allow dynamic source and target

language selection.
FR-8: BLEU score ≥ 30 for supported language pairs.

Model Options:

- MarianMT
- mBART
- Custom Transformer model

Libraries:

- transformers
- torch
- sentencepiece
- datasets

4.3 Text-to-Speech Module

FR-9: System shall convert translated text to speech.
FR-10: System shall support natural voice synthesis.
FR-11: Response time < 3 seconds.

Libraries:

- Coqui TTS
- gTTS
- pyttsx3

4.4 User Interface

FR-12: System shall provide language dropdown selection.
FR-13: System shall show original and translated text.
FR-14: System shall allow audio playback.
FR-15: System shall allow file upload (.wav, .mp3).

---

5. Non-Functional Requirements

5.1 Performance

- End-to-end latency ≤ 5 seconds
- Support 1000 concurrent users (scalable architecture)

5.2 Security

- HTTPS encryption
- JWT-based authentication
- Secure API keys

5.3 Scalability

- Microservices architecture
- Horizontal scaling via Kubernetes

5.4 Reliability

- 99.5% uptime
- Automatic model fallback

---

6. Technology Stack

6.1 Programming Languages

- Python (Backend + ML)
- JavaScript (Frontend)

6.2 ML Frameworks

- PyTorch
- HuggingFace Transformers

6.3 Backend

- FastAPI
- Uvicorn
- Gunicorn

6.4 Frontend

- React.js
- Tailwind CSS

6.5 DevOps & Deployment Platforms

Platform	Purpose
Amazon Web Services	Cloud hosting
Google Cloud	ML deployment
Microsoft Azure	AI services
Docker	Containerization
Kubernetes	Orchestration
Heroku	Quick deployment
Vercel	Frontend deployment

7. API Design (Sample)

7.1 Transcription Endpoint

POST /api/transcribe

Input: Audio file

Output: JSON { text: "..." }

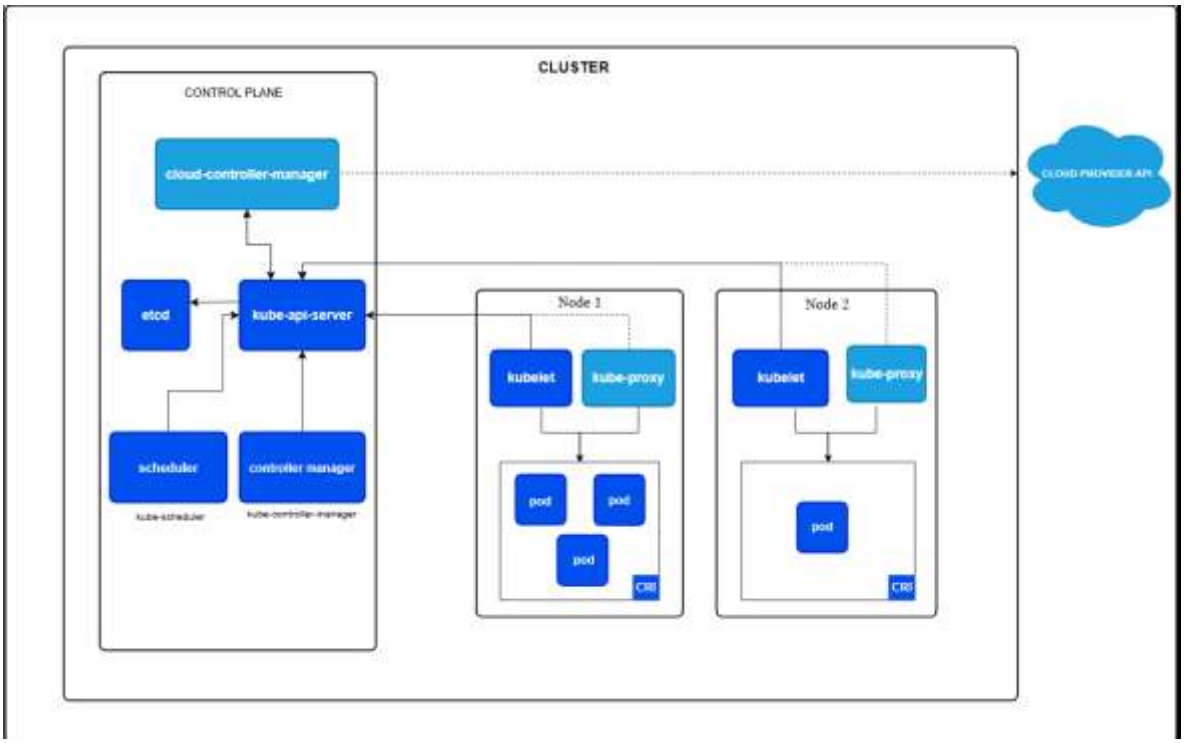
7.2 Translation Endpoint

POST /api/translate
Input: { source_lang, target_lang, text }
Output: { translated_text }

7.3 TTS Endpoint

POST /api/tts
Input: { text, language }
Output: Audio stream

8. Deployment Architecture



Deployment Steps

1. Containerize backend using Docker
2. Push image to container registry
3. Deploy via Kubernetes cluster
4. Configure CI/CD (GitHub Actions)
5. Enable auto-scaling

9. Database Design

Tables:

Users

- user_id
- email
- password_hash
- preferred_language

TranslationLogs

- log_id
- user_id
- source_text
- translated_text
- timestamp

---

10. Future Enhancements

- Real-time streaming translation
- Edge deployment (mobile device)

- Voice cloning
- Emotion-aware translation
- Offline mode
- **11. Risks & Mitigation**

Risk	Mitigation
High latency	Use model quantization
Low translation accuracy	Fine-tune Transformer
GPU cost	Auto-scaling
Noise in audio	Preprocessing + Noise filtering